Prompts

1 Summarising notes :

I am a B.Tech 2nd year student preparing for exams. Act as an expert academic assistant. Summarize the given notes and identify important concepts, definitions, and key points. Use a simple and easy-to-understand tone. Present the output using clear bullet points for quick revision.

Response:

Python Programming (R23) – Quick Revision Notes

Course Objectives

- Learn core concepts of Python programming.
- Understand Python data structures:
  - Lists
  - Tuples
  - Sets
  - Dictionaries
- Learn Functions, Modules, and Regular Expressions.
- Build practical Python applications.

UNIT–I: Python Basics

Important Concepts

- History and applications of Python.
- Installing Anaconda and Jupyter Notebook.
- Python syntax and structure.

Key Definitions

- **Identifier:** Name given to variables, functions, classes, etc.
- **Keyword:** Reserved words with special meaning (if, for, while, etc.).
- **Variable:** Stores data values.

- **Data Type:** Defines the type of data (int, float, str, etc.).
- **Operator:** Performs operations on values.
- **Indentation:** Spaces used to define code blocks.
- **Type Conversion:** Converting one data type into another.
- **Dynamic Typing:** Variable type decided at runtime.

Control Flow Statements

- if
- if-else
- if-elif-else
- Nested if
- while loop
- for loop
- break
- continue
- try-except (Exception Handling)

Important Operators

- Arithmetic Operators
- Relational Operators
- Assignment Operators
- Logical Operators
- Bitwise Operators
- Ternary Operator
- Membership Operators
- Identity Operators

Frequently Asked Programs

- Largest among three numbers.
- Prime numbers in an interval.
- Swapping two numbers without temporary variable.
- Multiplication table.
- Complex number operations.

UNIT–II: Functions, Strings and Lists

Functions

Definition

A **function** is a reusable block of code that performs a specific task.

Important Topics

- Built-in Functions
- User-defined Functions
- Function Definition and Calling
- Return Statement
- Void Function
- Scope and Lifetime of Variables
- Default Arguments
- Keyword Arguments
- *args
- **kwargs
- Command Line Arguments

Strings

Definition

A **string** is a sequence of characters enclosed in quotes.

Important Topics

- Creating Strings
- String Operations
- Indexing
- Slicing
- Joining Strings
- String Methods
- String Formatting

Lists

Definition

A **list** is an ordered and mutable collection of items.

Important Topics

- Creating Lists
- List Operations
- Indexing and Slicing
- Built-in Functions
- List Methods
- del Statement

Frequently Asked Programs

- Function with multiple return values.
- Function using default arguments.
- Length of string without library functions.
- Check substring existence.
- List insertion, addition, slicing.
- Demonstrate built-in list functions.

UNIT–III: Dictionaries, Tuples and Sets

Dictionaries

Definition

A **dictionary** stores data in key-value pairs.

Important Topics

- Creating Dictionaries
- Accessing Values
- Modifying Values
- Dictionary Methods
- Built-in Functions
- del Statement

Tuples

Definition

A **tuple** is an ordered but immutable collection.

Important Topics

- Tuple Creation
- Tuple Operations
- tuple() Function
- Indexing and Slicing
- Built-in Functions
- Relation between Lists and Tuples
- Relation between Dictionaries and Tuples
- zip() Function

Sets

Definition

A **set** is an unordered collection of unique elements.

Important Topics

- Set Creation
- Set Methods
- Frozenset

Frequently Asked Programs

- Concatenate tuples.
- Count vowels in a string.
- Check key existence in dictionary.
- Add key-value pair to dictionary.
- Sum all values in dictionary.

UNIT–IV: Files and Object-Oriented Programming

File Handling

Definition

File handling is used to store and retrieve data permanently.

Important Topics

- Types of Files
- Creating and Reading Text Files
- Writing Files
- Binary Files
- Pickle Module
- CSV Files
- os Module
- os.path Module

Object-Oriented Programming (OOP)

Definition

OOP is a programming paradigm based on objects and classes.

Important Concepts

Class

Blueprint for creating objects.

Object

Instance of a class.

Constructor

Special method used to initialize objects.

Encapsulation

Wrapping data and methods together.

Inheritance

One class acquires properties of another.

Polymorphism

Same method behaves differently for different objects.

Frequently Asked Programs

- Sort words from one file into another.
- Reverse file lines.
- Count characters, words and lines.
- Array operations.
- Matrix addition, transpose and multiplication.
- Shape class with area and perimeter methods.

UNIT–V: Introduction to Data Science

Functional Programming

- Uses functions as building blocks.
- Focuses on reusable and clean code.

JSON

Definition

JSON (JavaScript Object Notation) is a lightweight data-interchange format.

Important Uses

- Data storage
- Data transfer between applications

XML

Definition

XML (Extensible Markup Language) is used to store and transport structured data.

NumPy

Definition

NumPy is a Python library used for numerical computations and array operations.

Important Concepts

- Array Creation
- ndim
- shape
- size
- dtype
- Array Slicing
- Integer Indexing
- Boolean Indexing
- min()
- max()
- sum()
- cumulative sum

Pandas

Definition

Pandas is a Python library used for data analysis and manipulation.

Important Concepts

- DataFrame
- Creating DataFrames
- head() Function
- Data Selection Operations

Matplotlib

Definition

Matplotlib is used for data visualization.

Important Graphs

- Scatter Plot
- Line Plot

Frequently Asked Programs

- Check JSON string complexity.
- Create NumPy arrays.
- Demonstrate ndim, shape, size, dtype.
- Array slicing and indexing.
- Min, max, sum of arrays.
- Create DataFrame from dictionary.
- Apply head() and data selection operations.
- Scatter and plot visualizations.

Most Important Exam Topics (High Priority ⭑)

- Python Data Types
- Operators
- Control Flow Statements
- Functions and Arguments
- Strings and String Methods
- Lists, Tuples, Sets, Dictionaries
- File Handling
- OOP Concepts
    - Class
    - Object
    - Constructor
    - Inheritance
    - Polymorphism
    - Encapsulation
- JSON and XML

- NumPy Arrays
- Pandas DataFrames
- Matplotlib Graphs
- Exception Handling
- Matrix Operations
- Common Python Programs and Output-Based Questions

Last-Minute Revision Formula:
Python Basics → Functions → Strings → Lists → Dictionaries → Tuples → Sets → File Handling → OOP → JSON/XML → NumPy → Pandas → Matplotlib.

Why it works :

Because this prompt is in a structured format of "CREATE" :

Context: I am a B.Tech 2nd year student preparing for exams.

Role: Act as an expert academic assistant.

Action: Summarize the given notes and identify important concepts, definitions, and key points.

Tone: Simple and easy to understand.

Expected Format: Use clear headings and bullet points for quick revision.

2 Generating a study plan :

I want to learn Python programming in one week. Act as an experienced study mentor. Create a 7-day study plan covering Python basics, practice exercises, and revision tasks. Use a motivating and student-friendly tone. Present the study plan in a clear day-wise table with columns for Day, Topics to Learn, Practice Exercises, and Revision Tasks.

Response:

Response:

Day	Topics to Learn	Practice Exercises	Revision Tasks
Day 1	Introduction to Python, Installation, Variables, Data Types, Input/Output Statements	Write programs to display messages, take user input, and perform basic arithmetic operations	Revise variables, data types, and syntax rules
Day 2	Operators (Arithmetic, Relational, Logical), Type Conversion, Strings	Create a calculator, compare numbers, perform string operations like concatenation and slicing	Review operators and string functions
Day 3	Conditional Statements (if, if-else, elif, nested if`)	Check odd/even numbers, find the largest of three numbers, create a grade calculator	Revise all decision-making statements with examples
Day 4	Loops (for, while), break, continue, pass	Print patterns, generate multiplication tables, calculate factorials and sums	Review loop syntax and practice tracing loop execution
Day 5	Lists, Tuples, Sets, Dictionaries	Perform list operations, store student records, create simple dictionary-based programs	Revise data structure methods and differences
Day 6	Functions, Arguments, Return Values, Scope of Variables	Write functions for mathematical operations, palindrome checking, and simple utilities	Review function definitions and program flow
Day 7	File Handling Basics, Exception Handling, Mini Project, Overall Revision	Create a student record file program, handle errors using try-except, build a small project	Revise all topics, solve mixed practice questions, and review weak areas

Motivation Tips

- Spend **2–3 hours daily** on learning and practice.
- Follow the **Learn → Practice → Revise** cycle every day.
- Write code yourself instead of only reading examples.
- Debug errors patiently—they are part of learning.
- At the end of Day 7, build a small project such as a **Student Management System**, **Calculator**, or **To-Do List App** to strengthen your understanding.

Consistency for 7 days is more important than studying for long hours on a single day. 🚀

Why it works :

Because this prompt is in a structured format of "CREATE" :

Context: I want to learn Python programming in one week.

Role: Act as an experienced study mentor.

Action: Create a 7-day study plan covering Python basics, practice exercises, and revision tasks.

Tone: Motivating and student-friendly.

Expected Format: Present the study plan in a clear day-wise table with columns for Day, Topics to Learn, Practice Exercises, and Revision Tasks.

3 Creating interview questions :

I am preparing for an Agentic AI Developer interview. Act as a technical interviewer and generate 15 Agentic AI Developer interview questions along with their answers and detailed explanations. Cover important topics such as AI agents, tool calling, planning and reasoning, memory systems, Retrieval-Augmented Generation (RAG), agent workflows, Large Language Models (LLMs), prompt engineering, multi-agent systems, evaluation techniques, deployment strategies, monitoring, security, and scalability. Maintain a professional and educational tone throughout. Present the content in a continuous format where each interview question is immediately followed by its answer and explanation, without using side headings, tables, or separate section labels.

Response:

1. What is an AI agent, and how is it different from a traditional chatbot?

Answer: An AI agent is a software system that can perceive information, make decisions, plan actions, use tools, and achieve specific goals autonomously. A traditional chatbot mainly responds to user inputs based on conversation context.

Explanation: While chatbots focus on generating responses, AI agents can perform tasks such as searching the web, accessing databases, calling APIs, executing code, and interacting with external systems. Agentic AI extends the capabilities of Large Language Models by enabling them to act rather than only generate text.

1. What is tool calling in Agentic AI?

Answer: Tool calling is the ability of an AI agent to invoke external functions, APIs, databases, or software tools to complete tasks beyond its internal knowledge.

Explanation: For example, an AI agent can call a weather API to retrieve real-time weather information or query a database for customer records. Tool calling improves accuracy, extends functionality, and enables interaction with real-world systems.

1. Why is planning important in AI agents?

Answer: Planning helps an AI agent break down complex tasks into smaller, manageable steps before execution.

Explanation: Instead of directly answering a complicated query, an agent first identifies objectives, determines required actions, gathers information, and executes tasks in sequence. This improves reliability and task completion rates in multi-step workflows.

1. What is the difference between reasoning and planning?

Answer: Reasoning involves analyzing information and making logical decisions, while planning involves organizing actions required to achieve a goal.

Explanation: For example, reasoning helps determine which solution is most appropriate, whereas planning decides the sequence of steps needed to implement that solution. Effective AI agents often combine both capabilities.

1. What are memory systems in Agentic AI?

Answer: Memory systems allow AI agents to store and retrieve information from previous interactions or external knowledge sources.

Explanation: Short-term memory stores information relevant to the current conversation, while long-term memory preserves important user preferences, historical interactions, and learned experiences. Memory improves personalization and task continuity.

1. What is Retrieval-Augmented Generation (RAG)?

Answer: Retrieval-Augmented Generation is a technique that combines information retrieval with Large Language Models to generate more accurate and up-to-date responses.

Explanation: Instead of relying solely on model training data, RAG retrieves relevant documents from external knowledge sources and provides them as context to the LLM. This reduces hallucinations and improves factual accuracy.

1. What are the main components of a RAG pipeline?

Answer: The main components are document ingestion, text chunking, embedding generation, vector storage, retrieval, and response generation.

Explanation: Documents are converted into vector embeddings and stored in a vector database. When a query arrives, relevant documents are retrieved and passed to the LLM, which generates an informed response based on the retrieved context.

1. What is an agent workflow?

Answer: An agent workflow is the sequence of actions an AI agent follows to complete a task.

Explanation: A workflow may include understanding user intent, retrieving information, planning actions, invoking tools, validating outputs, and delivering results. Structured workflows increase consistency and reliability in agent execution.

1. How do Large Language Models contribute to Agentic AI systems?

Answer: Large Language Models act as the reasoning and decision-making engine of Agentic AI systems.

Explanation: They interpret user requests, generate plans, decide when to use tools, summarize information, and communicate results. However, LLMs alone are not agents until they are integrated with memory, tools, and execution frameworks.

1. What is prompt engineering, and why is it important?

Answer: Prompt engineering is the process of designing effective instructions that guide LLM behavior.

Explanation: Well-designed prompts improve response quality, reduce ambiguity, and ensure consistent outputs. Techniques such as role prompting, chain-of-thought prompting, few-shot prompting, and structured prompting are widely used in Agentic AI applications.

1. What are multi-agent systems?

Answer: Multi-agent systems consist of multiple AI agents working together to solve a problem.

Explanation: Each agent can specialize in a particular role, such as research, planning, coding, or validation. Collaboration among specialized agents often leads to better performance on complex tasks compared to a single agent.

1. How can Agentic AI systems be evaluated?

Answer: Agentic AI systems can be evaluated using metrics such as task success rate, accuracy, latency, cost, tool utilization efficiency, and user satisfaction.

Explanation: Evaluation should focus on both the final output and the decision-making process. Developers often use benchmark datasets, simulation environments, and human feedback to measure agent performance.

1. What deployment strategies are commonly used for AI agents?

Answer: Common deployment strategies include cloud deployment, containerization, microservices architecture, and serverless computing.

Explanation: Containers such as Docker simplify packaging and deployment, while orchestration platforms like Kubernetes help manage scalability and reliability. Cloud platforms provide infrastructure for running AI agents efficiently.

1. Why is monitoring important in Agentic AI systems?

Answer: Monitoring ensures that AI agents operate correctly, efficiently, and safely in production environments.

Explanation: Developers monitor metrics such as response time, token usage, API failures, hallucination rates, tool execution success, and user feedback. Continuous monitoring helps detect issues early and maintain system quality.

1. What security and scalability challenges exist in Agentic AI?

Answer: Security challenges include prompt injection attacks, unauthorized tool access, data leakage, and malicious inputs. Scalability challenges include handling large user volumes, managing memory efficiently, and optimizing computational resources.

Explanation: To address security risks, developers implement authentication, authorization, input validation, sandboxing, and audit logging. For scalability, techniques such as load balancing, distributed architectures, caching, vector database optimization, and horizontal scaling are commonly used. These measures ensure that Agentic AI systems remain secure, reliable, and capable of supporting growing workloads.

These are the types of HR-focused Agentic AI interview questions that are commonly asked to B.Tech AIML students and freshers during internship and entry-level hiring processes.

Why it works :

Because this prompt is in a structured format of "CREATE" :

Context: I am preparing for an Agentic AI Developer interview.

Role: Act as a technical interviewer.

Action: Generate 15 Agentic AI Developer interview questions with answers and explanations covering AI agents, tool calling, planning, memory, RAG, workflows, LLMs, prompt engineering, multi-agent systems, and deployment concepts.

Tone: Professional and educational.

Expected Format: Present each item with separate Question, Answer, and Explanation sections.

4 Writing a resume summary :

I am B.Tech CSE student applying for intenships.Act as an resume expert. Write a professional resume summary highlighting technical skills, projects, and career goals. Maintain a professional and confidential tone throughout the resume. Present a single paragraph of 100 to 150 words.

Response:

RESUME

Name: [Your Full Name]
Phone: [Your Phone Number]
Email: [Your Email Address]
LinkedIn: [LinkedIn Profile URL]
GitHub: [GitHub Profile URL]
Location: [City, State]

Career Objective

Motivated and enthusiastic B.Tech Computer Science Engineering student seeking an internship opportunity to apply technical knowledge, problem-solving abilities, and programming skills in a professional environment. Eager to gain hands-on experience in software development, learn emerging technologies, and contribute effectively to organizational goals while enhancing my technical and professional growth.

Education

Bachelor of Technology (B.Tech) – Computer Science Engineering
[College Name]

Expected Graduation: [Year]
CGPA: [CGPA]

Intermediate (Class XII)
[College Name]
Year of Completion: [Year]
Percentage: [Percentage]

SSC (Class X)
[School Name]
Year of Completion: [Year]
Percentage: [Percentage]

Technical Skills

- Programming Languages: Python, Java, C
- Web Technologies: HTML, CSS, JavaScript
- Database: MySQL
- Tools: Git, GitHub, VS Code
- Concepts: Data Structures, Object-Oriented Programming (OOP), DBMS

Projects

Student Management System

- Developed a system to manage student records efficiently.
- Implemented features for adding, updating, deleting, and viewing student details.

Personal Portfolio Website

- Designed and developed a responsive portfolio website using HTML, CSS, and JavaScript.
- Showcased projects, skills, and contact information.

Certifications

- Python Programming Certification
- Introduction to Data Structures
- Web Development Fundamentals
- Database Management Systems (DBMS)

Soft Skills

- Communication Skills
- Teamwork and Collaboration
- Problem-Solving
- Time Management
- Adaptability
- Quick Learning Ability

Languages Known

- English
- Telugu
- Hindi

Interests

- Software Development
- Artificial Intelligence
- Competitive Programming
- Learning New Technologies
- Reading Technical Blogs

Declaration:
I hereby declare that the above information is true and correct to the best of my knowledge and belief.

Place: _____
Date: _____
Signature: _____

Why it works :

Because this prompt is in a structured format of "CREATE" :

Context: I am a B.Tech CSE student applying for internships.

Role: Act as an HR recruiter and resume expert.

Action: Write a professional resume summary highlighting technical skills, projects, and career goals.

Tone: Professional and confident.

Expected Format: A single paragraph of 100–150 words.

5 Creating a lesson plan :

I need to teach Python loops to beginner students. Act as an experienced computer science teacher and create a lesson plan that explains Python loops in a simple and engaging way. Include clear learning objectives so students understand the purpose of loops, practical classroom activities to help them practice, real-world examples that demonstrate how loops are used in programming, and assessment questions to evaluate their understanding. Use a clear, beginner-friendly teaching style and organize the response into sections for Objectives, Activities, Examples, and Assessment.

Response:

Lesson Plan: Introduction to Python Loops

Objectives

By the end of this lesson, students will be able to:

- Understand what a loop is and why loops are used in programming.
- Differentiate between for loops and while loops.
- Write simple Python programs using loops.
- Use loops to repeat tasks efficiently.
- Identify real-world situations where loops are useful.

Activities

Activity 1: Everyday Loop Examples (5 Minutes)

Ask students:

- What do you do every day repeatedly?
    - Brushing teeth
    - Walking to class
    - Counting steps
    - Practicing exercises

Discussion: Explain that computers also perform repetitive tasks, and loops help automate them.

Activity 2: First Python Loop (10 Minutes)

```
for i in range(5):
    print("Hello Students!")
```

Output:

```
Hello Students!
Hello Students!
Hello Students!
Hello Students!
Hello Students!
```

Explanation:

- range(5) generates numbers from 0 to 4.
- The loop repeats the print statement 5 times.

Activity 3: Counting Numbers (10 Minutes)

```
for number in range(1, 11):
    print(number)
```

Task for Students:

- Modify the program to print numbers from 1 to 20.
- Print only even numbers from 2 to 20.

Activity 4: While Loop Practice (10 Minutes)

```
count = 1

while count <= 5:
    print(count)
    count += 1
```

Explanation:

- The loop continues as long as the condition is true.
- count += 1 increases the value each time.

Student Challenge:

- Change the program to count from 10 down to 1.

Activity 5: Mini Coding Challenge (10 Minutes)

Ask students to write a program that:

1. Prints their name 5 times.

2. Displays numbers from 1 to 10.
3. Calculates the sum of numbers from 1 to 5.

Example:

```
total = 0

for i in range(1, 6):
    total += i

print("Sum =", total)
```

Real-World Examples

Example 1: Attendance System

```
students = ["Rahul", "Priya", "Anu"]

for student in students:
    print(student)
```

Use: Displaying all student names.

Example 2: Sending Notifications

```
for i in range(3):
    print("Notification Sent")
```

Use: Sending repeated notifications.

Example 3: Password Attempts

```
attempts = 1

while attempts <= 3:
```

```
    print("Try Again")
    attempts += 1
```

Use: Limiting login attempts.

Example 4: Shopping Cart Items

```
cart = ["Laptop", "Mouse", "Keyboard"]

for item in cart:
    print(item)
```

Use: Displaying products in an online shopping cart.

Assessment

Multiple Choice Questions

1. What is the purpose of a loop?

A. Store data
B. Repeat instructions
C. Create variables
D. Delete files

Answer: B. Repeat instructions

---

2. Which loop is used when the number of repetitions is known?

A. while loop
B. for loop
C. if statement
D. function

Answer: B. for loop

Short Answer Questions

1. What is a loop in Python?

Answer: A loop is a programming structure used to execute a block of code repeatedly.

---

2. What is the difference between a for **loop and a** while **loop?**

Answer:

- for loop: Used when the number of repetitions is known.
- while loop: Used when repetition depends on a condition.

Practical Questions

1. Write a program to print numbers from 1 to 5.

```
for i in range(1, 6):
    print(i)
```

---

2. Write a program to print your name 3 times.

```
for i in range(3):
    print("Durga Priya")
```

Lesson Summary

- Loops help automate repetitive tasks.
- Python provides two main loops:
    - for loop → repeats a fixed number of times.
    - while loop → repeats until a condition becomes false.

- Loops are widely used in games, websites, mobile apps, attendance systems, and data processing.
- Practice is the key to mastering loops in Python.

Why it works :

Because this prompt is in a structured format of "CREATE" :

Context: I need to teach Python loops to beginner students.

Role: Act as an experienced computer science teacher.

Action: Create a lesson plan including objectives, activities, examples, and assessments.

Tone: Clear and engaging.

Expected Format: Sections for Objectives, Activities, Examples, and Assessment.